

PostgreSQL

Complete Coursebook

From Zero to Professional — A Hands-On Visual Guide

- 17 comprehensive modules with 18+ visual diagrams
- Real-world examples you can run immediately
- Edge cases, pitfalls, and best practices
- JOIN diagrams, B-tree visualizations, query flow charts
- Practice exercises with solutions

Table of Contents

Module 1: Getting Started with PostgreSQL

Module 2: Databases

Module 3: Schemas

Module 4: Tables & Data Types

Module 5: Constraints

Module 6: INSERT, UPDATE, DELETE (DML)

Module 7: Querying Data (SELECT)

Module 8: Aggregations & GROUP BY

Module 9: Joins

Module 10: Subqueries & CTEs

Module 11: Views

Module 12: Indexes & Performance

Module 13: Window Functions

Module 14: Transactions

Module 15: Built-in Functions

Module 16: Users, Roles & Permissions

Module 17: Practice Projects & Next Steps

Module 1: Getting Started with PostgreSQL

What is PostgreSQL?

PostgreSQL (also called **Postgres**) is a free, open-source **relational database management system** (RDBMS). It stores data in tables made of rows and columns — like a super-powered spreadsheet that can safely handle millions of rows.

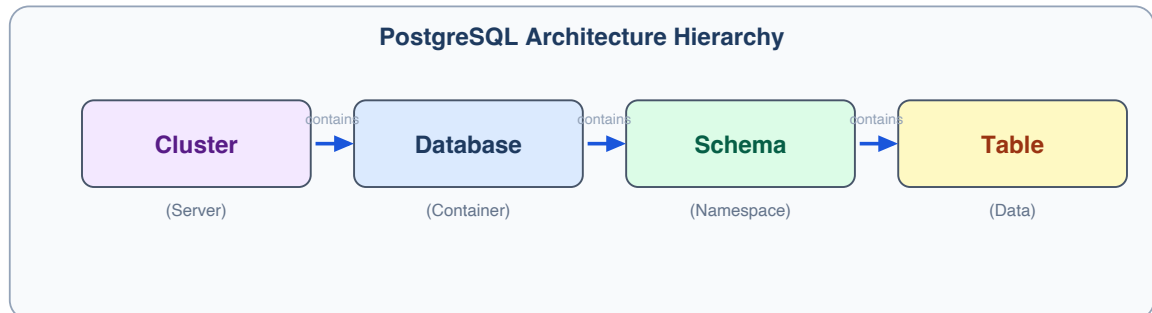


Fig 1: PostgreSQL organizes data in a clear hierarchy

How PostgreSQL Compares

Database	Type	Cost	Best For
PostgreSQL	Relational	Free	Web apps, analytics, general use
MySQL	Relational	Free	Simple web apps, WordPress
SQLite	Relational	Free	Mobile apps, small projects
MongoDB	Document	Free	Flexible schema needs
Oracle	Relational	Paid	Enterprise systems

Installing PostgreSQL

```
# Ubuntu/Debian
sudo apt update && sudo apt install postgresql

# Docker (recommended for learning)
docker run --name my-postgres \
  -e POSTGRES_PASSWORD=mypass \
  -p 5432:5432 -d postgres
```

Connecting

```
# Command line
psql -U postgres -h localhost

# You'll see: postgres=# (you're connected!)
```

Essential psql Commands

Command	What It Does	Example
\l	List all databases	Shows every database
\c dbname	Switch to a database	\c shop_db
\dt	List tables	Shows table names
\d tablename	Describe table structure	\d users
\dn	List schemas	Shows schema names
\du	List users/roles	Shows access info
\di	List indexes	Shows index names
\x	Toggle vertical display	Readable wide rows
\q	Quit psql	Exit

Module 2: Databases

Understanding the Hierarchy

Cluster = The whole PostgreSQL server. **Database** = An isolated container within the cluster. **Schema** = A folder inside a database. **Table** = Where data lives.

Creating, Switching, Dropping

```
CREATE DATABASE library_db;
\l                -- list databases
\c library_db     -- switch to it
DROP DATABASE IF EXISTS library_db;
```

Warning: DROP DATABASE

1. You CANNOT drop a database you're connected to (switch first with `\c postgres`).
2. You CANNOT drop if others are connected. Disconnect them first.
3. It's PERMANENT with no undo. Always back up first.

Force-Dropping

```
-- Terminate all other connections first
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'library_db' AND pid != pg_backend_pid();

DROP DATABASE library_db;
```

Database Ownership

```
CREATE DATABASE shop_db OWNER alice;
ALTER DATABASE shop_db OWNER TO bob;
GRANT CONNECT ON DATABASE shop_db TO analyst_role;
```

Module 3: Schemas

What is a Schema?

A schema is a **namespace** (logical folder) inside a database. It prevents name conflicts: `sales.orders` and `archive.orders` are different tables.

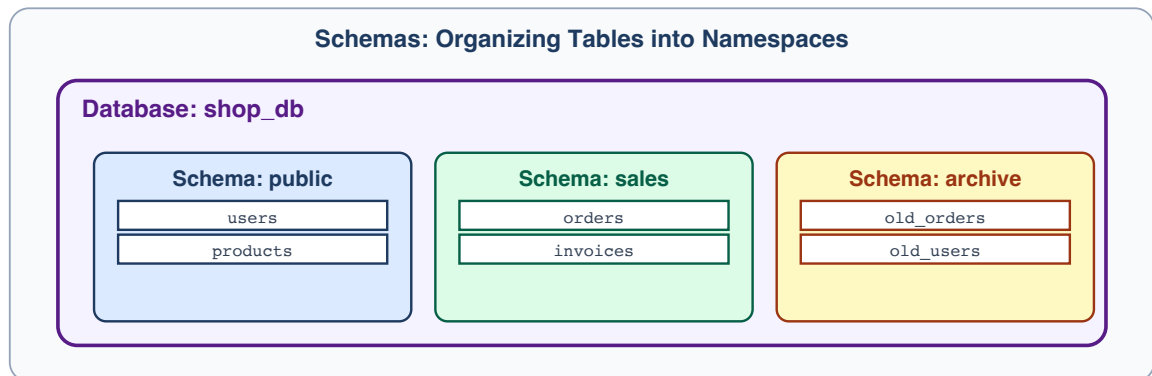


Fig 2: Schemas organize tables within a database

Working with Schemas

```
CREATE SCHEMA sales;
CREATE SCHEMA IF NOT EXISTS archive;
\dn -- list schemas

-- Create table in a schema
CREATE TABLE sales.orders (id SERIAL PRIMARY KEY, total NUMERIC);

-- Access table with full path
SELECT * FROM sales.orders;
```

Schema Search Path

```
SHOW search_path; -- default: "$user", public
SET search_path TO sales, public;
-- Now unqualified names check 'sales' first
```

Dropping a Schema

```
DROP SCHEMA sales; -- fails if objects exist
DROP SCHEMA sales CASCADE; -- drops everything inside!
```

Module 4: Tables & Data Types

Anatomy of a PostgreSQL Table

Columns (Fields)					Rows (Records)
id (PK)	name	email	salary	dept_id (FK)	
1	Alice	alice@co.com	85000	10	
2	Bob	bob@co.com	72000	20	
3	Eve	eve@co.com	91000	10	
4	Dave	NULL	68000	30	

NULL = Unknown/Missing

Fig 3: Anatomy of a PostgreSQL table with rows, columns, and NULL

Creating a Table

```
CREATE TABLE books (  
  id SERIAL PRIMARY KEY,  
  title TEXT NOT NULL,  
  author VARCHAR(100),  
  published_date DATE,  
  price NUMERIC(10, 2),  
  is_available BOOLEAN DEFAULT TRUE,  
  tags TEXT[],  
  metadata JSONB,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

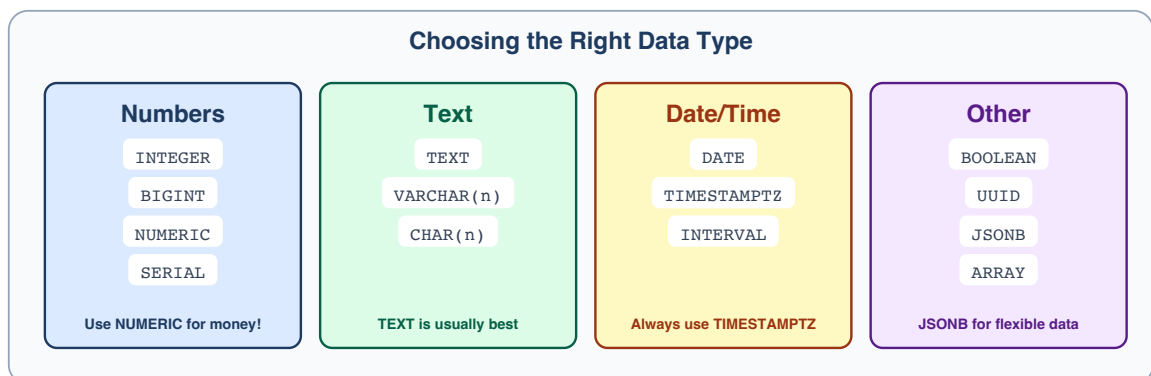


Fig 4: Choosing the right data type for each use case

Common Data Types

Type	Size	Use For	Example
INTEGER	4 bytes	Most IDs and counts	42

BIGINT	8 bytes	Very large numbers	9000000000
SERIAL	4 bytes	Auto-increment PK	1, 2, 3...
NUMERIC(p,s)	Variable	Money, exact decimals	199.99
TEXT	Unlimited	Any string (preferred)	'Hello'
VARCHAR(n)	Max n chars	Length-limited strings	'abc'
BOOLEAN	1 byte	Flags	TRUE / FALSE
DATE	4 bytes	Date only	2026-04-16
TIMESTAMPZ	8 bytes	Date + time + TZ	Always use this!
JSONB	Variable	Flexible data	'{"key": "val"}'
UUID	16 bytes	Globally unique ID	a0eebc99-...
TEXT[]	Variable	Array of strings	'{"a","b"}'

Critical: Money Values

NEVER use REAL or DOUBLE PRECISION for money (rounding errors!). Always use NUMERIC(p,s).

Best Practice

Always use TIMESTAMPZ instead of TIMESTAMP. It stores in UTC and converts automatically.

Altering a Table

```
ALTER TABLE books ADD COLUMN isbn VARCHAR(20);
ALTER TABLE books DROP COLUMN isbn;
ALTER TABLE books RENAME COLUMN author TO author_name;
ALTER TABLE books ALTER COLUMN price TYPE NUMERIC(12, 2);
ALTER TABLE books RENAME TO library_books;
```

Dropping and Temporary Tables

```
DROP TABLE IF EXISTS books;
DROP TABLE books CASCADE; -- also drops dependent views

-- Temp table: exists only for your session
CREATE TEMP TABLE scratch AS SELECT * FROM books WHERE price > 500;
```


Module 5: Constraints

Constraints are **rules** the database enforces automatically. They are your last line of defense against bad data.

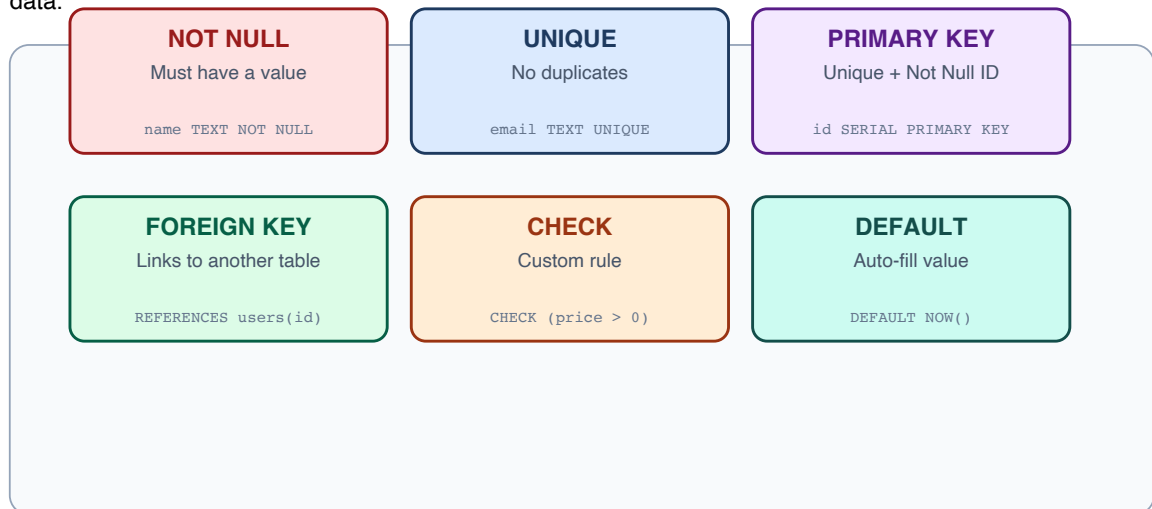


Fig 5: Six types of constraints in PostgreSQL

1. NOT NULL

```
CREATE TABLE users (  
    name TEXT NOT NULL,      -- required  
    phone TEXT               -- optional (NULL allowed)  
);  
-- INSERT INTO users (phone) VALUES ('123'); -- ERROR: name is required
```

2. UNIQUE

```
email TEXT UNIQUE          -- no duplicate emails  
-- NOTE: UNIQUE allows multiple NULLs!  
-- Combine with NOT NULL to prevent: email TEXT UNIQUE NOT NULL
```

3. PRIMARY KEY

```
-- Single column  
id SERIAL PRIMARY KEY  
  
-- Composite (two columns together are unique)  
PRIMARY KEY (student_id, course_id)  
  
-- Modern alternative  
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

4. FOREIGN KEY

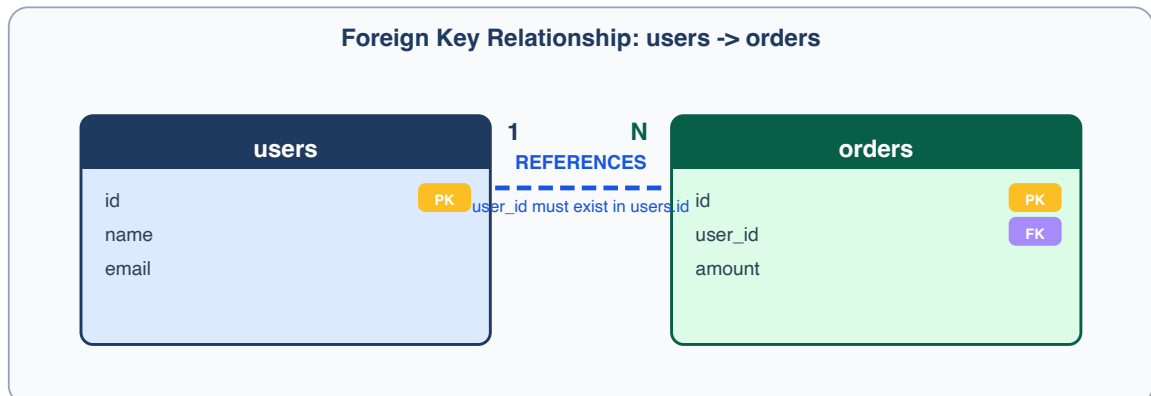


Fig 6: Foreign key links orders.user_id to users.id

```
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE
);
-- user_id MUST match an existing users.id
```

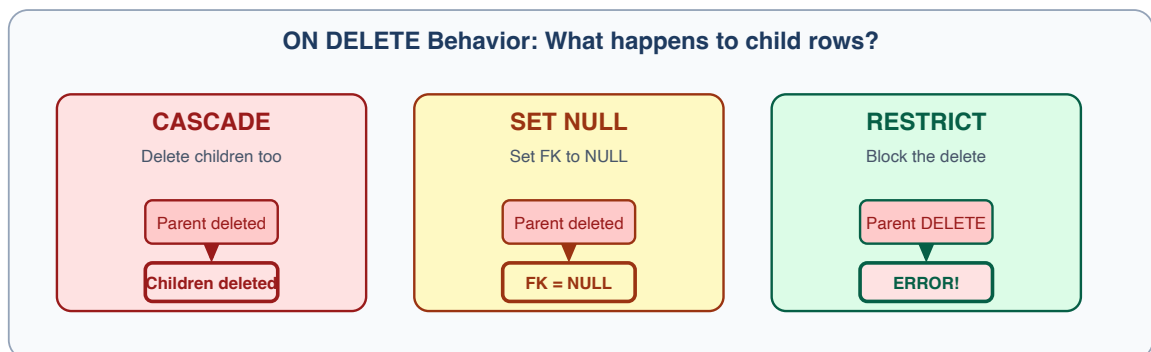


Fig 7: What happens to child rows when parent is deleted

Important: Index Your Foreign Keys!

PostgreSQL does NOT auto-create indexes on FK columns. Always add: `CREATE INDEX idx_orders_user_id ON orders(user_id);`

5. CHECK

```
price NUMERIC CHECK (price > 0)
status TEXT CHECK (status IN ('pending', 'shipped', 'delivered'))
-- Multi-column: CHECK (end_date >= start_date)
```

6. DEFAULT

```
status TEXT DEFAULT 'draft'
view_count INT DEFAULT 0
created_at TIMESTAMPTZ DEFAULT NOW()
```

Altering Constraints

Constraint	Can Alter Directly?	How to Change
NOT NULL	Yes	ALTER COLUMN col SET/DROP NOT NULL
DEFAULT	Yes	ALTER COLUMN col SET/DROP DEFAULT
CHECK	No	DROP CONSTRAINT + ADD CONSTRAINT
UNIQUE	No	DROP CONSTRAINT + ADD CONSTRAINT
PRIMARY KEY	No	DROP CONSTRAINT + ADD PRIMARY KEY
FOREIGN KEY	No	DROP CONSTRAINT + ADD CONSTRAINT

```
-- Safe pattern: transaction
BEGIN;
  UPDATE products SET price = 50 WHERE price <= 50;
  ALTER TABLE products DROP CONSTRAINT positive_price;
  ALTER TABLE products ADD CONSTRAINT positive_price CHECK (price >= 50);
COMMIT;
```

Module 6: INSERT, UPDATE, DELETE

INSERT

```
-- Single row
INSERT INTO books (title, author, price)
VALUES ('1984', 'George Orwell', 299.00);

-- Multiple rows
INSERT INTO books (title, author, price) VALUES
('Dune', 'Frank Herbert', 499),
('Foundation', 'Isaac Asimov', 399);

-- From another table
INSERT INTO archived_books (title, author)
SELECT title, author FROM books WHERE published_date < '2000-01-01';
```

UPDATE

```
UPDATE books SET price = price * 1.10 WHERE author = 'Orwell';
```

Critical

Always use WHERE with UPDATE. Without it, ALL rows get changed!

DELETE

```
DELETE FROM books WHERE id = 5;
-- DELETE FROM books; -- DANGER: deletes EVERYTHING!
```

RETURNING — See What Changed

```
INSERT INTO books (title) VALUES ('New Book') RETURNING id, title;
UPDATE books SET price = 199 WHERE id = 3 RETURNING *;
DELETE FROM books WHERE id = 5 RETURNING title;
```

UPSERT (INSERT ... ON CONFLICT)

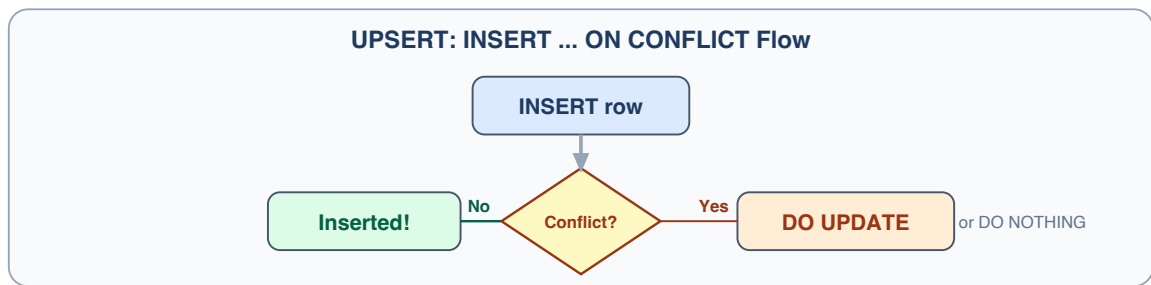


Fig 8: UPSERT decision flow — insert or update based on conflict

```
INSERT INTO books (id, title, price) VALUES (1, 'Dune', 550)
ON CONFLICT (id)
DO UPDATE SET price = EXCLUDED.price;
-- EXCLUDED = the row you tried to insert

-- Silently skip duplicates:
ON CONFLICT (id) DO NOTHING;
```

Module 7: Querying Data (SELECT)

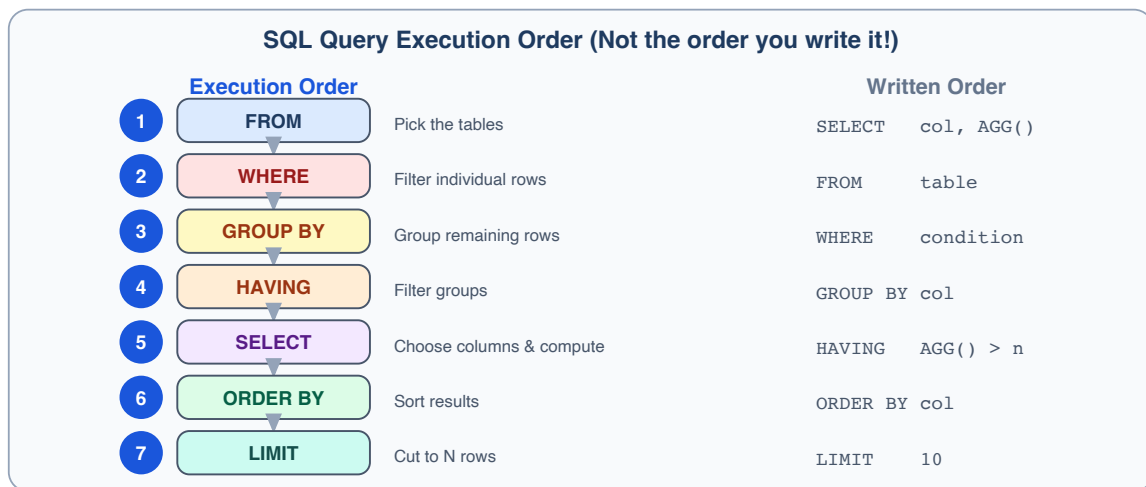


Fig 9: SQL runs in a different order than you write it!

Basic SELECT

```
SELECT * FROM books;           -- all columns
SELECT title, price FROM books; -- specific columns
SELECT title AS name FROM books; -- alias
```

WHERE, AND, OR, NOT

```
SELECT * FROM books WHERE price > 300;
SELECT * FROM books WHERE price > 200 AND is_available = TRUE;
SELECT * FROM books WHERE price < 100 OR price > 900;
SELECT * FROM books WHERE NOT is_available;
```

Comparison Operators

Operator	Meaning	Example
=, !=, <, >	Basic comparison	WHERE price > 300
BETWEEN	Range (inclusive)	WHERE price BETWEEN 100 AND 500
IN	Match any in list	WHERE author IN ('Orwell','Asimov')
LIKE / ILIKE	Pattern (% = any, _ = one char)	WHERE title ILIKE 'the %'

NULL Handling

How NULL Behaves (Common Gotchas)

5 + NULL = NULL

NULL = NULL = NULL (not TRUE!)

NULL != NULL = NULL (not TRUE!)

WHERE x = NULL -> No rows ever!

WHERE x IS NULL -> Correct way!

COALESCE(NULL, 0) = 0

Fig 10: NULL behaves unexpectedly — know these rules!

```
SELECT * FROM books WHERE author IS NULL;      -- correct
SELECT * FROM books WHERE author = NULL;       -- WRONG!
SELECT COALESCE(author, 'Unknown') FROM books; -- replace NULL
```

ORDER BY, LIMIT, DISTINCT

```
SELECT * FROM books ORDER BY price DESC;
SELECT * FROM books ORDER BY price ASC NULLS LAST;
SELECT * FROM books ORDER BY id LIMIT 10 OFFSET 20; -- pagination
SELECT DISTINCT author FROM books;
```

Module 8: Aggregations & GROUP BY

Aggregate Functions

```
SELECT COUNT(*) FROM books;           -- all rows
SELECT COUNT(author) FROM books;      -- non-NULL only!
SELECT COUNT(DISTINCT author) FROM books;
SELECT SUM(price), AVG(price), MIN(price), MAX(price) FROM books;
```

COUNT(*) vs COUNT(column)

COUNT(*) counts ALL rows including NULLs. COUNT(column) counts only non-NULL values. This is a common bug source.

GROUP BY

```
SELECT author, COUNT(*) AS book_count
FROM books GROUP BY author;

SELECT author, genre, AVG(price) AS avg_price
FROM books GROUP BY author, genre;
```

Rule

Every column in SELECT must be in GROUP BY or inside an aggregate function.

HAVING — Filter Groups

```
SELECT author, COUNT(*) AS cnt
FROM books
GROUP BY author
HAVING COUNT(*) > 3;  -- only authors with 3+ books
```

Feature	WHERE	HAVING
Runs	Before grouping	After grouping
Filters	Individual rows	Groups
Can use aggregates?	No	Yes

STRING_AGG and ARRAY_AGG

```
SELECT author, STRING_AGG(title, ', ') FROM books GROUP BY author;
-- Orwell | 1984, Animal Farm

SELECT author, ARRAY_AGG(title) FROM books GROUP BY author;
-- Orwell | {1984, "Animal Farm"}
```


Module 9: Joins

A JOIN combines rows from two or more tables based on a related column.

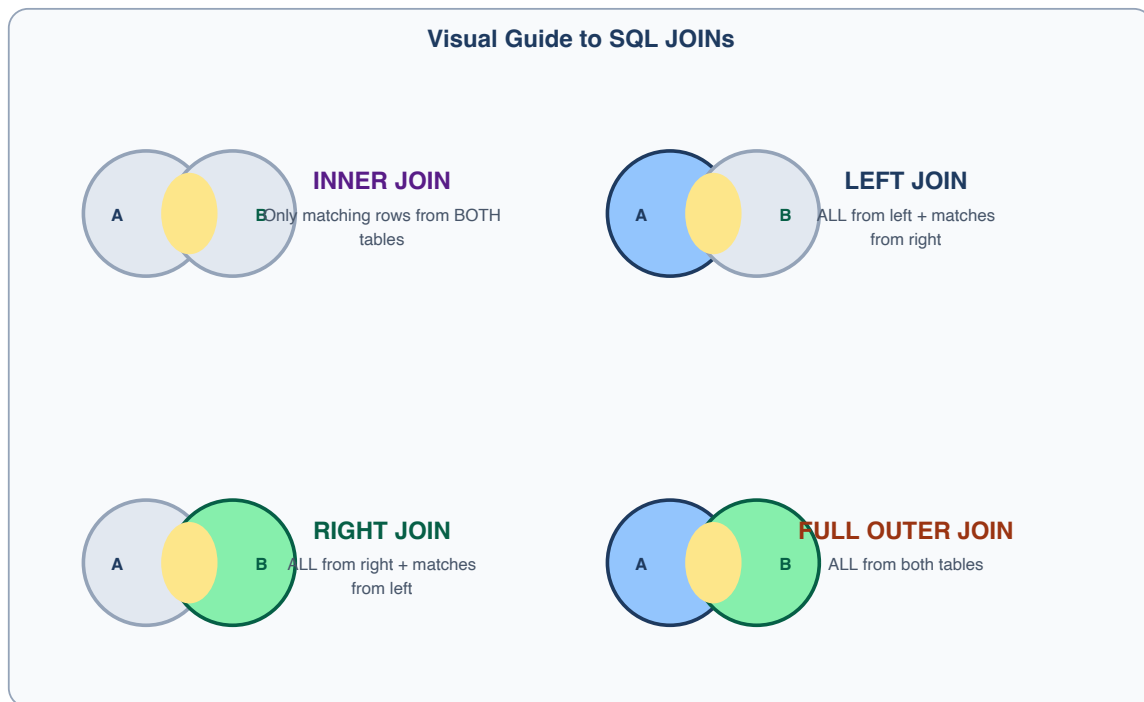


Fig 11: Visual Venn diagrams for all JOIN types

INNER JOIN — Only Matching Rows

```
SELECT u.name, o.amount
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
-- Only users WITH orders appear
```

LEFT JOIN — All Left + Matches

```
SELECT u.name, o.amount
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
-- ALL users appear; no orders = NULL
```

Common Pitfall

Adding `WHERE o.column = 'x'` to a LEFT JOIN silently turns it into an INNER JOIN (NULLs fail the WHERE). Move the condition into the ON clause instead.

Other JOINS

```
-- RIGHT JOIN: All from right table + matches from left
-- FULL OUTER JOIN: All from both, NULLs where no match
```

```
-- CROSS JOIN: Every row x every row (Cartesian product)
-- SELF JOIN: Table joined with itself (hierarchies)
```

Multi-Table Joins

```
SELECT u.name, o.id, p.title
FROM users u
JOIN orders o ON u.id = o.user_id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id;
```

Best Practice

Always use table aliases (u, o, p) and qualify columns (u.name not just name) when joining.

Module 10: Subqueries & CTEs

Subqueries

IN Subquery

```
SELECT * FROM books
WHERE author_id IN (
    SELECT id FROM authors WHERE country = 'UK'
);
```

EXISTS (Often Faster)

```
SELECT * FROM authors a
WHERE EXISTS (SELECT 1 FROM books b WHERE b.author_id = a.id);
```

Correlated Subquery

```
-- Books priced above their author's average
SELECT title, price FROM books b1
WHERE price > (SELECT AVG(price) FROM books b2 WHERE b2.author = b1.author);
```

CTEs (WITH Clause)

CTEs are named temporary result sets. Cleaner than nested subqueries.

```
WITH expensive AS (
    SELECT * FROM books WHERE price > 500
)
SELECT author, COUNT(*) FROM expensive GROUP BY author;
```

Multiple CTEs

```
WITH
    active AS (SELECT * FROM users WHERE is_active),
    recent AS (SELECT * FROM orders WHERE created_at > NOW() - '30 days'::INTERVAL)
SELECT a.name, COUNT(r.id)
FROM active a JOIN recent r ON a.id = r.user_id
GROUP BY a.name;
```

Recursive CTE (Hierarchies)

```
WITH RECURSIVE subordinates AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id, s.level + 1
    FROM employees e JOIN subordinates s ON e.manager_id = s.id
)
SELECT * FROM subordinates ORDER BY level;
```

Infinite Loop Risk

Always add a depth limit: `WHERE s.level < 10`

Module 11: Views

A view is a **saved query** you can use like a table. It doesn't store data (except materialized views).

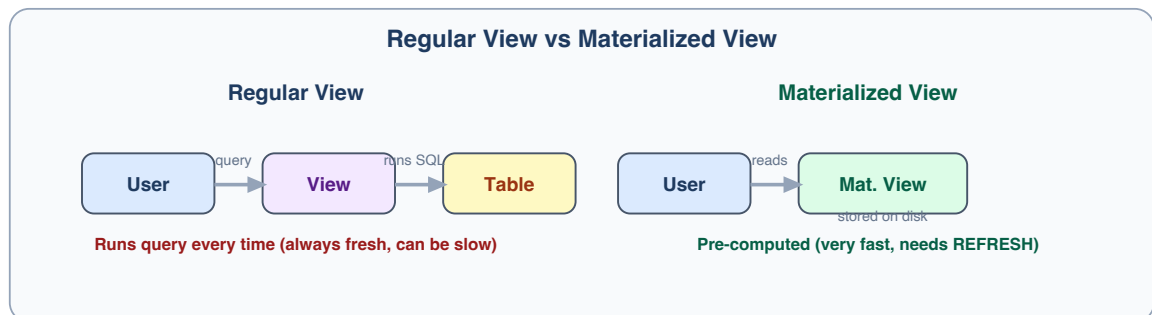


Fig 12: Regular View (always fresh) vs Materialized View (fast, needs refresh)

```
-- Regular view
CREATE VIEW expensive_books AS
SELECT title, author, price FROM books WHERE price > 500;
SELECT * FROM expensive_books; -- runs the query each time

-- Materialized view (stores result on disk)
CREATE MATERIALIZED VIEW book_stats AS
SELECT author, COUNT(*) AS total, AVG(price) AS avg_price
FROM books GROUP BY author;

REFRESH MATERIALIZED VIEW book_stats; -- update stale data
```

Feature	Regular View	Materialized View
Stores data?	No (query each time)	Yes (on disk)
Speed	Same as underlying query	Very fast (pre-computed)
Freshness	Always current	Stale until refreshed
Indexable?	No	Yes

Module 12: Indexes & Performance

An index is a lookup structure that makes searches fast. Without one, Postgres reads every row. With one, it jumps directly.

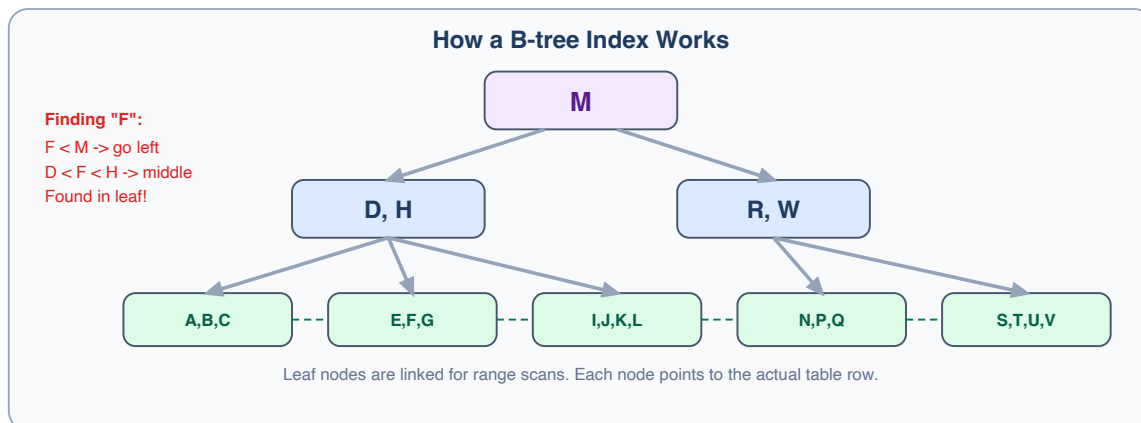


Fig 13: How a B-tree index organizes data for fast lookup

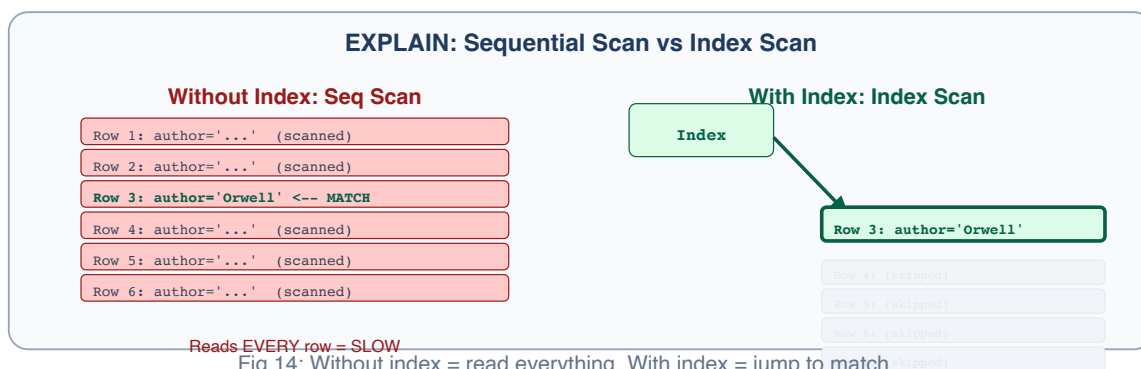


Fig 14: Without index = read everything. With index = jump to match.

Jumps to match = FAST

Creating Indexes

```
CREATE INDEX idx_books_author ON books(author);
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Partial index (only matching rows)
CREATE INDEX idx_active ON users(email) WHERE is_active = TRUE;

-- GIN index for JSONB/arrays
CREATE INDEX idx_tags ON books USING GIN(tags);
```

Composite Index — Left-to-Right Rule

Composite Index: Left-to-Right Rule

INDEX on (author, published_date, price)

OK	WHERE author = 'X'	Uses index (leftmost column)
OK	WHERE author = 'X' AND published_date > '2020'	Uses index (A, then B)
OK	WHERE author = 'X' AND published_date > '2020' AND price < 500	Uses full index (A, B, C)
X	WHERE published_date > '2020'	Cannot use! B without A
X	WHERE price < 500	Cannot use! C without A, B

Fig 15: Composite indexes only help queries starting from the leftmost column

EXPLAIN — See the Query Plan

```
EXPLAIN SELECT * FROM books WHERE author = 'Orwell';  
-- Shows plan without running
```

```
EXPLAIN ANALYZE SELECT * FROM books WHERE author = 'Orwell';  
-- Runs query and shows real execution time
```

What You See	Meaning
Seq Scan	Reading every row (bad on big tables)
Index Scan	Using an index (good)
Index Only Scan	Index has all data needed (best)
Hash Join	Efficient join for medium data
Nested Loop	Join method for small datasets

Module 13: Window Functions

Window functions compute across a set of rows **without collapsing them**.

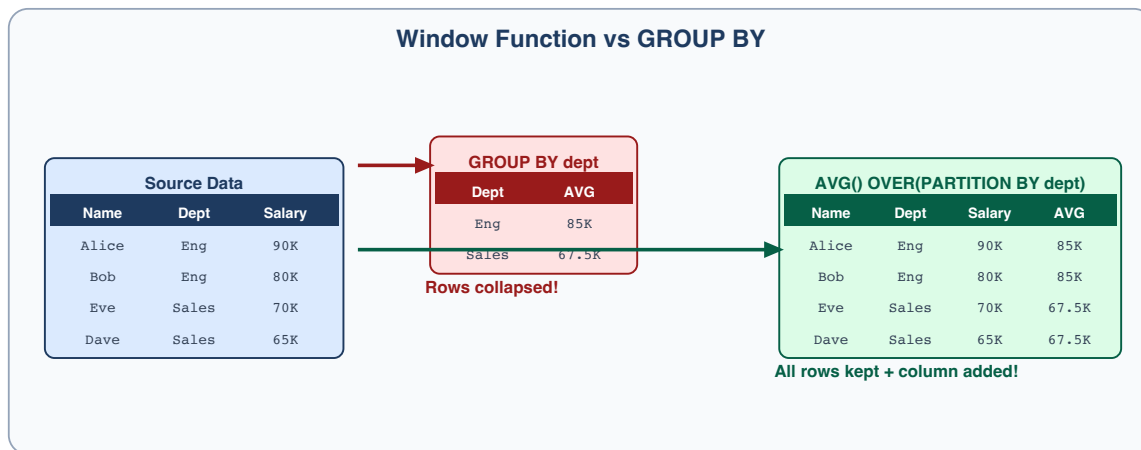


Fig 16: GROUP BY collapses rows. Window functions keep all rows and add a column.

ROW_NUMBER, RANK, DENSE_RANK

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

Name	Salary	ROW_NUMBER	RANK	DENSE_RANK
Alice	90K	1	1	1
Bob	80K	2	2	2
Eve	80K	3	2	2
Dave	70K	4	4	3

PARTITION BY

```
-- Rank within each department
SELECT name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;
```

LAG/LEAD — Previous/Next Row

```
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change
FROM monthly_sales;
```


Running Totals

```
SELECT order_date, amount,  
       SUM(amount) OVER (ORDER BY order_date) AS running_total  
FROM orders;
```

Module 14: Transactions

A transaction is a group of operations that succeed or fail **together**.

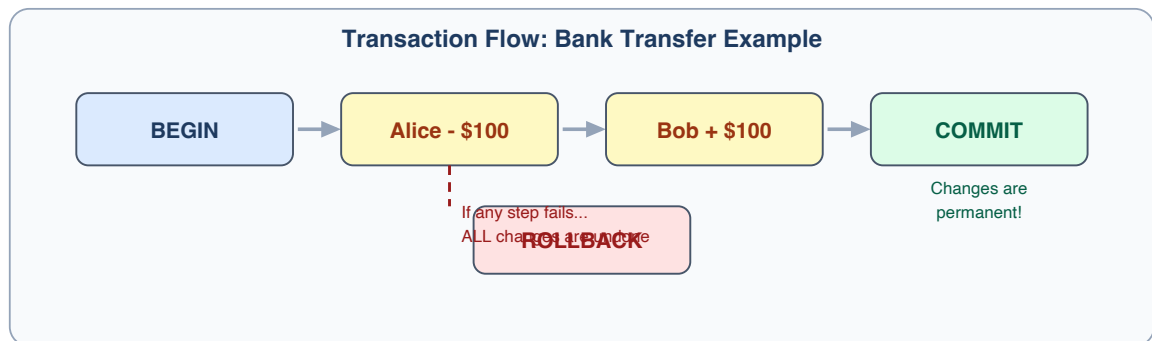


Fig 17: Transaction flow — all steps succeed (COMMIT) or all are undone (ROLLBACK)

ACID Properties

Property	Meaning	Simple Explanation
Atomicity	All-or-nothing	ALL steps succeed, or NONE do
Consistency	Valid state always	Database rules never broken
Isolation	No interference	Concurrent transactions don't clash
Durability	Permanent once committed	Survives crashes

BEGIN, COMMIT, ROLLBACK

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;  -- save changes
-- or ROLLBACK; -- undo everything
```

Savepoints

```
BEGIN;
INSERT INTO orders (...);
SAVEPOINT before_items;
INSERT INTO order_items (...);  -- oops
ROLLBACK TO before_items;      -- undo only this step
INSERT INTO order_items (...);  -- retry
COMMIT;
```

Isolation Levels

Level	What You See	Use When
-------	--------------	----------

READ COMMITTED (default)	Only committed data	Most apps
REPEATABLE READ	Snapshot at transaction start	Reports
SERIALIZABLE	As if running one-by-one	Financial operations

Common Mistake

In psql, if a query errors inside a transaction, ALL subsequent commands fail. Fix: run ROLLBACK, then start over.

Module 15: Built-in Functions

String Functions

```
CONCAT('Hello', ' ', 'World')      -- Hello World
UPPER('hello') / LOWER('HELLO')    -- HELLO / hello
LENGTH('hello')                    -- 5
SUBSTRING('PostgreSQL', 1, 4)       -- Post
REPLACE('apple pie', 'apple', 'cherry') -- cherry pie
SPLIT_PART('a,b,c', ',', 2)        -- b
TRIM('  hello  ')                  -- hello
LEFT('PostgreSQL', 4)               -- Post
```

Date/Time Functions

```
NOW()                               -- current timestamp
CURRENT_DATE                        -- today
EXTRACT(YEAR FROM NOW())             -- 2026
DATE_TRUNC('month', NOW())           -- first of month
AGE('2000-01-01'::DATE)              -- 26 years ...
NOW() + INTERVAL '7 days'           -- 7 days from now
```

Type Casting

```
'123'::INTEGER                      -- text -> int
NOW()::DATE                          -- timestamp -> date
CAST('123' AS INTEGER)               -- standard SQL syntax
```

JSON/JSONB Functions

```
data->'user'                         -- get JSON value (returns JSON)
data->>'user'                         -- get text value (returns TEXT)
data @> '{"score":42}'               -- contains check
jsonb_set(data, '{score}', '100')   -- update field
```

CASE Expression

```
SELECT title, price,
       CASE
         WHEN price < 100 THEN 'Budget'
         WHEN price < 500 THEN 'Mid-range'
         ELSE 'Premium'
       END AS category
FROM books;
```

UNION / INTERSECT / EXCEPT

```
SELECT name FROM customers UNION SELECT name FROM suppliers;  
-- UNION removes dupes; UNION ALL keeps them  
-- INTERSECT: rows in both  
-- EXCEPT: rows in first but not second
```

Module 16: Users, Roles & Permissions

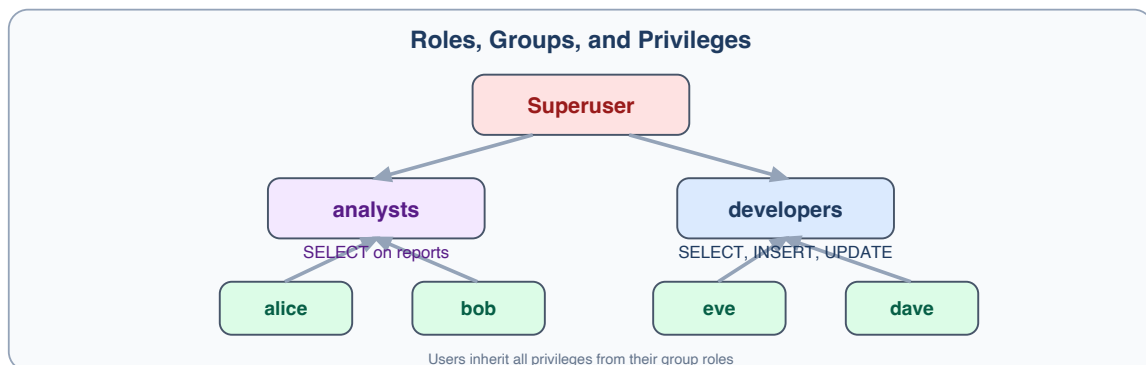


Fig 18: Roles, groups, and inherited privileges

Creating Roles

```
CREATE ROLE alice LOGIN PASSWORD 'secret'; -- a user
CREATE ROLE analysts; -- a group (no login)
```

GRANT and REVOKE

```
GRANT SELECT ON books TO alice;
GRANT SELECT, INSERT, UPDATE ON books TO alice;
GRANT ALL PRIVILEGES ON books TO alice;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analysts;
REVOKE INSERT ON books FROM alice;
```

Schema Privileges

```
-- Must grant USAGE before table access works!
GRANT USAGE ON SCHEMA sales TO alice;
GRANT SELECT ON ALL TABLES IN SCHEMA sales TO alice;
```

Common Error

Granting SELECT on a table but forgetting USAGE on its schema causes: ERROR: permission denied for schema

Role Membership (Groups)

```
GRANT analysts TO alice; -- alice inherits analyst privileges
REVOKE analysts FROM alice; -- remove from group
```

Diagnosing Permission Errors

```
\du alice -- show role info
\dp books -- show privileges on table
SELECT * FROM information_schema.role_table_grants
WHERE grantee = 'alice';
```

Module 17: Practice Projects & Next Steps

Practice Project: Task Tracker

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL, email TEXT UNIQUE NOT NULL  
);  
CREATE TABLE projects (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL, owner_id INT REFERENCES users(id)  
);  
CREATE TABLE tasks (  
    id SERIAL PRIMARY KEY,  
    title TEXT NOT NULL,  
    status TEXT DEFAULT 'pending'  
    CHECK (status IN ('pending', 'done', 'cancelled')),  
    project_id INT REFERENCES projects(id) ON DELETE CASCADE,  
    assignee_id INT REFERENCES users(id),  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_tasks_assignee ON tasks(assignee_id);  
CREATE INDEX idx_tasks_project ON tasks(project_id);
```

10 Practice Queries

1. All tasks

```
SELECT * FROM tasks;
```

2. Pending only

```
SELECT * FROM tasks WHERE status = 'pending';
```

3. Newest first

```
SELECT * FROM tasks ORDER BY created_at DESC;
```

4. Count per status

```
SELECT status, COUNT(*) FROM tasks GROUP BY status;
```

5. Tasks + assignee (JOIN)

```
SELECT t.title, u.name  
FROM tasks t JOIN users u ON t.assignee_id = u.id;
```

6. Projects + task count

```
SELECT p.name, COUNT(t.id)  
FROM projects p LEFT JOIN tasks t ON p.id = t.project_id  
GROUP BY p.name;
```

7. Users with 2+ tasks

```
SELECT u.name, COUNT(t.id)  
FROM users u JOIN tasks t ON u.id = t.assignee_id  
GROUP BY u.name HAVING COUNT(t.id) > 1;
```

8. Assignee = project owner

```
SELECT t.title FROM tasks t
JOIN projects p ON t.project_id = p.id
WHERE t.assignee_id = p.owner_id;
```

9. Rank by task count

```
SELECT u.name, COUNT(t.id),
       RANK() OVER (ORDER BY COUNT(t.id) DESC)
FROM users u LEFT JOIN tasks t ON u.id = t.assignee_id
GROUP BY u.name;
```

10. Latest task per user

```
SELECT * FROM (
  SELECT u.name, t.title, t.created_at,
         ROW_NUMBER() OVER (PARTITION BY u.id
                             ORDER BY t.created_at DESC) AS rn
  FROM users u JOIN tasks t ON u.id = t.assignee_id
) sub WHERE rn = 1;
```

What to Learn Next

- **Triggers** — functions that run automatically on data changes
- **Stored Procedures** — reusable SQL/PL/pgSQL code in the database
- **Partitioning** — splitting huge tables for performance
- **Replication** — copying data to standby servers
- **VACUUM & ANALYZE** — maintenance for healthy databases
- **pg_dump / pg_restore** — backup and recovery

End of Coursebook

Practice daily. Run every example. Read error messages carefully.